



10

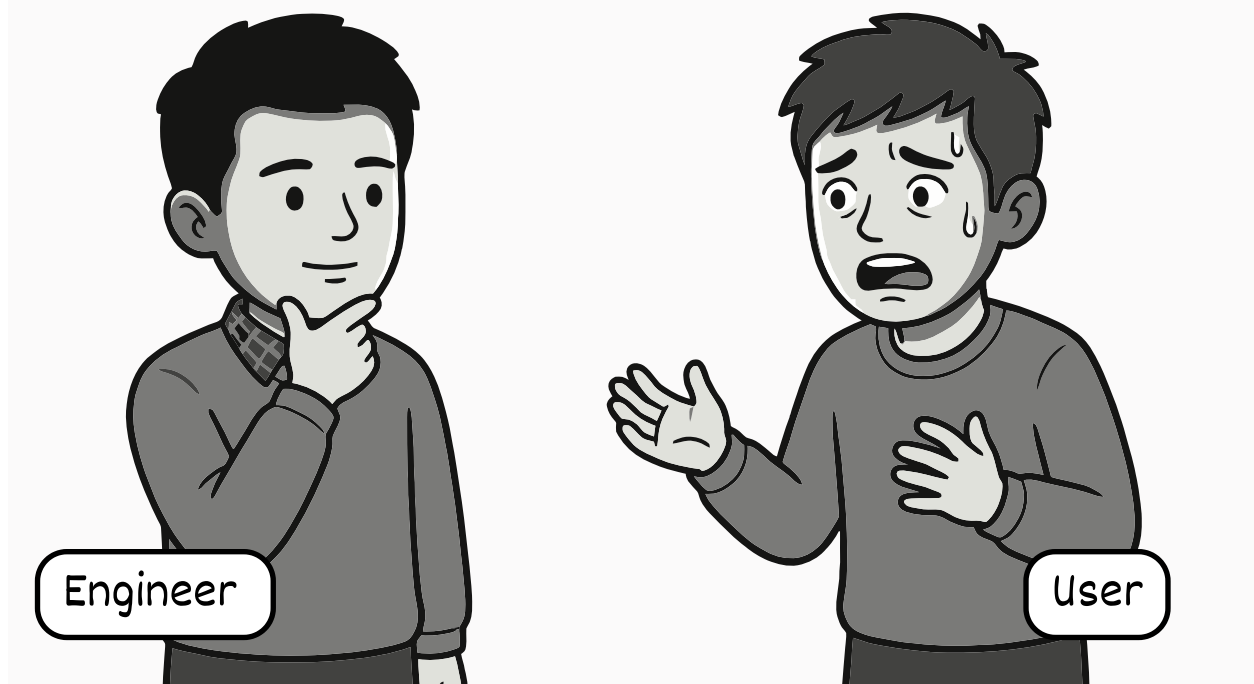
Customer and User Focus

Pierre, a mid-level data engineer, had helped build a self-service pipeline platform that had transformed how analysts accessed data. The old process had required filing tickets and waiting days for engineering support. Now, analysts could create their own extracts in hours, and leadership even cited the platform as proof that self-service tooling worked.

Three months after the launch, Pierre was optimizing pipeline performance when he noticed something odd in the usage logs. Analysts created pipelines, they ran them successfully, then they downloaded the results, but Pierre had access to broader system logs, and he saw a pattern emerging. The same analysts who had downloaded data had immediately spun up Python jobs that transformed those extracts before using them. They weren't using the pipeline outputs directly. They were treating them as raw material for additional processing.

Pierre brought this to his tech lead, who pointed out that ticket volume was down and, thus, the analysts were happy. As far as his tech lead was concerned, the platform worked as designed. But Pierre wondered whether "working as designed" simply meant "helping analysts succeed." During his regular work, he asked various analysts what happened after they downloaded their data. Their answers were consistent: they always had to pivot and aggregate the outputs before the data was in a

Customer and User Focus



Convincing his team to revisit a successful project wasn't easy. Pierre documented the post-download workflows he'd discovered and calculated the hours spent by analysts on repetitive transformations. He proposed adding output format options and common transformation templates, changes that fit within the existing architecture, rather than requiring a rebuild. His tech lead was skeptical at first, but the evidence of wasted analyst time made the case. After the improvements shipped, analysts started using pipeline outputs directly. The platform finally delivered on its promise of getting analysts to insights faster, not just data. Several analysts thanked Pierre directly, saying it was the first time engineering had truly understood their needs.

What Does It Mean to Focus on Customers and Users?

Customer focus means consistently turning user needs into effective technical solutions. It involves understanding why users need something and knowing how to meet that need.

Most people can implement features according to the specifications, but turning feature requests into insights about user needs is the mark of someone who is customer-focused. For frontend developers, it is someone who learns to identify potential friction points for users before they become complaints. For platform engineers, it is the one who studies how other teams use their APIs and improves the developer experience proactively. For data engineers, it's the one who understands how different teams consume data and will design pipelines that make their analyses easier.

looks for people who deliver "care," their term for deeply understanding and addressing user needs. Amazon's "Customer Obsession" principle states that leaders start with the customer and work backwards. These companies recognize that technical skills create lasting value only when they solve real problems for their customers.

The Essence of Customer and User Focus

Users rarely ask for what they actually need. They will request features based on their current pain, but they don't typically know to ask for better solutions. Customer focus means seeing past feature requests to see what users are trying to accomplish. That deeper understanding changes how you make technical decisions, prioritize your work, and measure success.

Consider a common scenario. Users will often request an export button because they need to analyze data in spreadsheets. You could add the button, job done. But if you ask the users why they're exporting, you might discover that they want to run calculations that your system could handle directly. In that case, the export request is a symptom of the real need: better analysis tools. Simply implementing every request without asking why will lead to cluttered systems full of workarounds. And ignoring user input entirely eventually leads to useless "solutions." Good customer focus means investigating the underlying problems behind requests and finding solutions that will address those real needs.

Customer focus also requires recognizing that users aren't a single group with uniform needs. They will have a range of technical abilities, and different segments will use your systems differently. And they will likely have conflicting priorities. Strong customer focus means deciding carefully which users to prioritize when you can't serve everyone equally. Do you optimize for power users versus beginners, internal teams versus external customers, or current users versus future growth? All these choices will have implications that you will need to understand before making a considered decision.

Customer focus includes measuring real outcomes that are often difficult to quantify. This means defining success from the user's perspective, tracking whether your solutions actually improve their outcomes, and iterating based on real use. The people who excel at customer focus want to know if their work truly helps users achieve their goals.

Cultural and Organizational Considerations

How an organization values and expresses customer focus will vary depending on its business model and user relationships. Some companies interact directly with their end users, while others build products where internal teams or business partners sit between them and the people who actually use their work. If you know these differences, you will know how to show your customer focus skills to interviewers at each type of company.

people who can interpret user behavior data and turn it into product improvements. Success comes from understanding mass user patterns while also understanding individual user journeys. The skill is balancing quantitative and qualitative data to create products that users will love.

Enterprise Software Companies (e.g., Salesforce, ServiceNow, Workday): These companies value people who understand complex business workflows and competing stakeholder needs. Customer focus here means managing end users who use the software daily and buyers who make purchasing decisions. Success in this sort of environment requires turning business requirements into technical solutions that enable the users of the system to work effectively.

Platform and Developer Tool Companies (e.g., AWS, MongoDB, Stripe): User focus at a platform or developer tool company means understanding technical users' workflows and accounting for the spectrum of use from occasional to power user. These companies value people who can think like their developer customers and are skilled at deciding which segments to prioritize. Success comes from recognizing that most users need only a fraction of available features, then creating tools that make common tasks simple while still providing depth for advanced use cases.

Internal Tools and Backend Teams: Customer focus here means understanding your impact on users even when you're far removed from them. Backend teams might serve frontend teams who serve customers, or build systems that affect users indirectly through performance and reliability. These teams need people who can trace how their technical decisions will affect the end users, whether they are customers or colleagues using internal tools. Success in this sort of context requires thinking beyond the immediate technical requirements and looking at the full chain of impact.

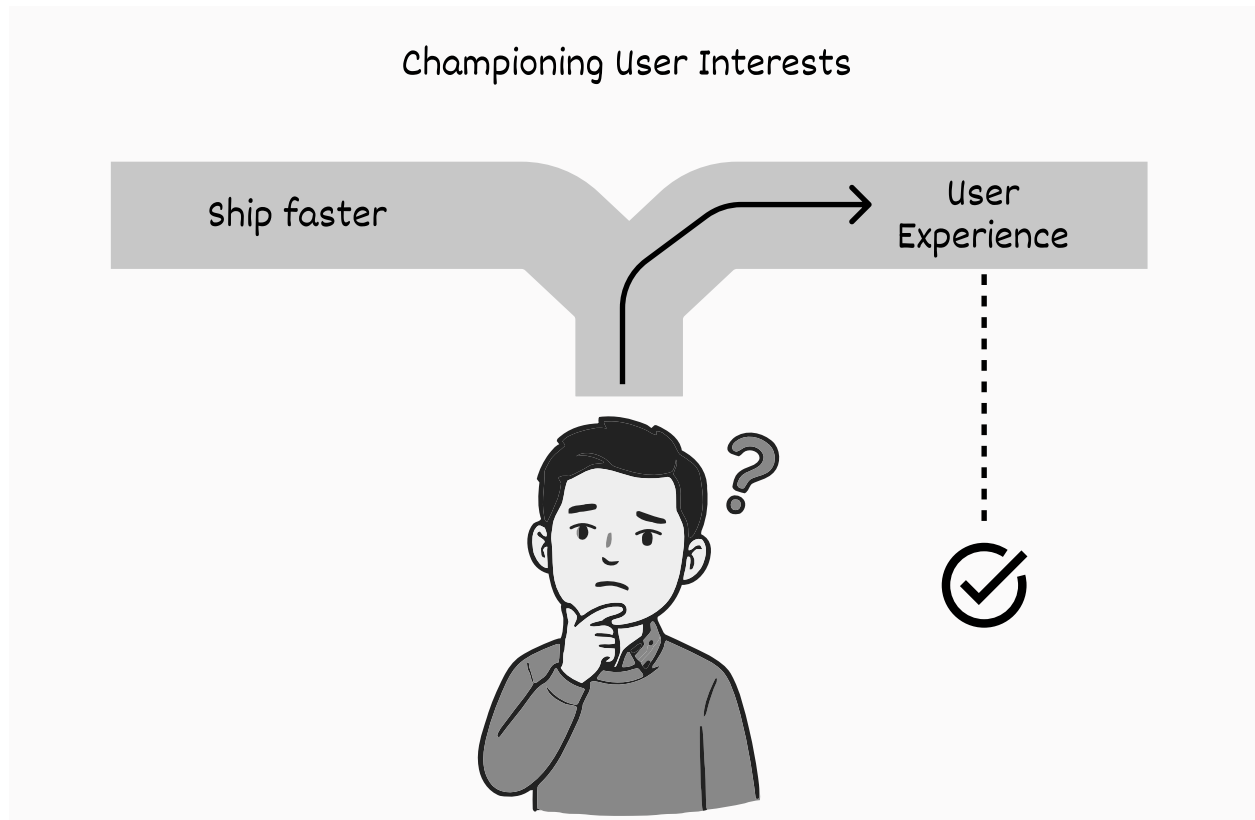
Cultural Background Considerations: Different cultures have varying approaches to learning about and serving customers. Some cultures emphasize explicit communication and direct feedback, while others rely on reading between the lines and anticipating unspoken needs. If your background emphasizes indirect communication, you might excel at noticing what users don't say. If you come from a culture that is known to favor being very direct, you might need to highlight how you've learned to probe deeper than surface requests.

Related Competencies

Customer Focus vs. Innovation: Innovation shows your ability to create new solutions. Customer focus makes sure that those solutions address real needs. You might invent an ergonomic new interface pattern (innovation), but perhaps it confuses users because it doesn't match their mental models. Customer focus keeps innovation grounded in what actually helps people succeed.

Customer Focus vs. Taking Initiative: Initiative involves identifying opportunities for improvement and taking action. Customer focus guides you to work on improvements that will actually matter to

Customer Focus vs. Problem-Solving: Problem-solving involves investigating issues thoroughly. Customer focus means understanding the problem from the user's perspective. Without customer focus, you might solve a speed issue (problem-solving) without realizing that the users care more about data accuracy than speed. Customer focus provides the context for deciding which problems will have the biggest impact for users.



Interview Questions

Interviewers explore your customer focus by asking questions that reveal your understanding of user needs and how to translate them into technical decisions. Do you care about what users actually need, or do you just do your job?

Understanding User Needs

- "Tell me about a time you had to understand user needs for a technical project."
- "Describe how you have approached building something when the user requirements weren't clear."
- "Give me an example of validating what users had actually needed versus what they had asked for."

These questions are asked to assess whether you will dig deeper to understand the true nature of user problems. Interviewers want to hear your description of the methods you use to uncover needs rather than just hearing you say that you accept feature requests without looking deeper. Strong answers will

Balancing Technical and User Needs

- "Give me an example of when user needs conflicted with technical best practices."
- "Describe a time you made technical trade-offs to better serve users."

Here, they're testing your ability to make well-considered trade-offs between competing priorities. These questions probe whether you can find creative solutions that achieve both technical quality and user success. Good examples will show you protecting critical user experiences while maintaining technical integrity, or advocating for a technical approach different from what was requested because it would better serve user needs, even if it means more work.

Measuring User Success

- "Describe how you measured whether your solution actually helped users."
- "Tell me about a time when you discovered users weren't benefiting from a feature you'd built."
- "How do you validate that your technical work improves user outcomes?"

Interviewers want to evaluate your ability to measure success from the user's perspective. The best stories will show you defining what success looks like for the user, gathering feedback about usage, and adjusting your approach based on user outcomes rather than just quantitative metrics.

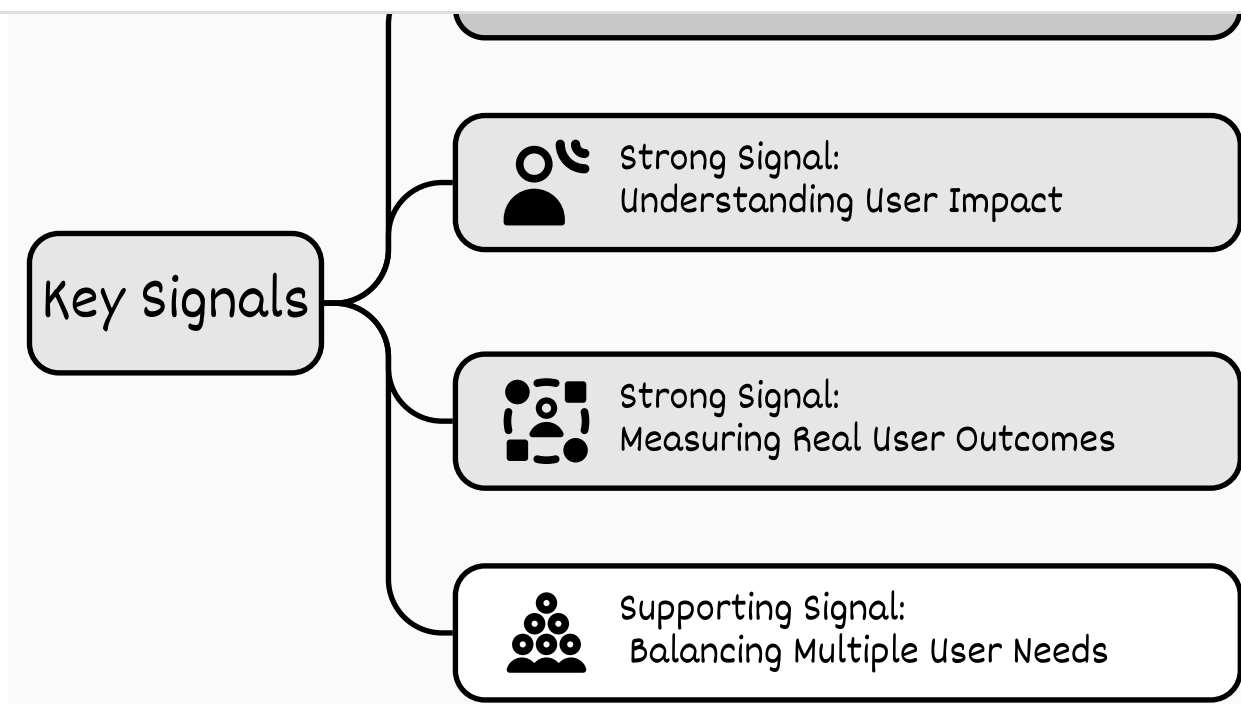
Advocating for Users

- "Tell me about a time when you advocated for users in a technical decision."
- "Describe a time when you pushed back on a technical approach because of user impact."
- "Give an example of you influencing technical decisions to better serve user needs."

These questions probe whether you will champion user interests even when doing so will create more work and inconvenience for you. Interviewers want to know if you help teams consider user impact in making technical choices. Strong responses will show you bringing the user's perspective to technical discussions and helping teams to consider technical and user trade-offs.

Key Signals

When evaluating customer focus, interviewers look for evidence that you can turn user needs into effective technical solutions. The strongest candidates show that they understand users deeply and make decisions that will genuinely help them succeed.



Critical Signal: User-Centered Decision-Making

Users struggle to explain their real challenges. They often don't understand how the software works, or they won't share their actual problems up front. Instead, they might request features that respond to the symptoms they're experiencing or relate to how the system has worked historically. Good stories will show that you probe deeper through questions, observation, and analysis to uncover the needs behind requests. You will show that you can turn those discoveries into solutions that will address what the users are really trying to accomplish.

For people working directly with end users, you should show that you can translate user goals into well-built software. For those working on internal tools, platform services, or backend systems, your "customer" might be other engineering teams, data scientists, or operations staff. Good examples in those areas will show you understanding those internal customers' workflows, the problems they're trying to solve using your systems, and how your technical choices can enable or hinder their success. Show the interviewers that you prioritize work based on user impact rather than your technical interests.

Strong Signal: Understanding User Impact

People with strong customer focus find ways to uncover user needs, even when they can't observe the end users directly. For customer-facing roles, this means watching users work with your systems, asking them questions that reveal their underlying needs, and testing whether your solutions will actually help.

who interact with users daily. You might track metrics that directly connect your technical work to user outcomes, for example, API response times that affect user experience, database query performance that impacts page load, or system reliability that determines whether users can complete their tasks. Your customers might be other engineering teams who depend on your services. Strong examples will show you proactively gathering feedback from these internal customers, seeing how your technical decisions cascade to end users, and prioritizing work based on downstream impact.

Strong Signal: Measuring Real User Outcomes

User-focused people define success from the user's perspective, not just from that of the system. This separates feature-focused engineers from user-focused ones. Good stories will show you instrumenting metrics that reflect the goals of the users, monitoring how users' behavior changes after releases, and iterating based on usage patterns. You measure task completion, goal achievement, and user efficiency, not just technical system metrics. The key is framing success in terms of what affects users.

Supporting Signal: Balancing Multiple User Needs

Strong customer focus means making thoughtful decisions about which user segments to prioritize. You can show that you understand the needs of power users versus casual users, can serve larger user segments without sacrificing smaller user segments, can balance advanced features for experts with simple tools for beginners, and can clearly articulate why you prioritized one group over another when you couldn't serve everyone equally.

Red Flags

We use Red Flags for serious issues that will significantly damage your candidacy and Yellow Flags for concerning patterns that will weaken your examples.

These patterns will weaken your customer focus stories.

Red Flag: Building Features Blindly

Implementing requested features without understanding why the users think they need them suggests a shallow customer focus. If your story jumps straight to implementation without thinking about how it will be used, you're demonstrating order-taking rather than problem-solving. Instead, show how you uncovered what the users were really trying to achieve.

Yellow Flag: Treating User Understanding as a One-Time Event

Checking user needs once at project kickoff and never revisiting them shows an incomplete customer focus. Your understanding of the users should evolve as you build and learn more. Stories that have

might have missed important use cases. Strong customer focus is shown by testing whether your solution works as you build it, not just at the beginning or end. Strong answers will show how you validated your assumptions at multiple points, adjusting your approach as you learned more about what the users needed.

Yellow Flag: Technical Optimization Without User Context

Improving technical metrics without connecting to user outcomes shows misplaced priorities. Making systems faster or more efficient only matters if it helps users accomplish their goals. If your improvements don't translate to better user experiences, you're optimizing for engineering satisfaction rather than customer success. Therefore, connect your technical improvements to specific user benefits.

Yellow Flag: One-Size-Fits-All Solutions

Treating all users the same demonstrates a shallow understanding of the diversity of users. Different user segments have different needs, workflows, and technical abilities. Stories that assume users are all the same will suggest that you don't think deeply about who uses your systems. Show the interviewers how you identified different user groups and made careful decisions about addressing their varied needs.

Example Stories by Level

Entry-Level Example: Internal Analytics Tool

Question: "Tell me about a time when you had to understand user needs for a technical project."

Headline: "I discovered that our sales team was barely using the analytics dashboard I'd spent weeks building."

Key Point 1: "I was assigned to add new metrics to our sales analytics dashboard. The requirements listed 15 different graphs and filters that the sales directors wanted implemented. But after deploying the features, our usage logs showed that adoption was really low, at around 20%. Instead of assuming that they simply needed training, I decided to sit with a few sales reps during their morning routine. I watched three different reps, and they all did the same thing. They'd log in, screenshot one specific number (their current quarter progress), and immediately close the dashboard. They never used any of the complex filtering I'd built."

Key Point 2: "When I asked why they only looked at one number, they explained that they checked the dashboard on their phones between customer calls. Our dashboard was desktop-optimized. It had tiny buttons and required horizontal scrolling on mobile. The filters I'd built required multiple clicks to get to personal metrics. I proposed building a simple mobile view that would show each rep their key

devices lasting under 30 seconds."

Key Point 3: "I built a mobile-first view that showed personalized metrics without any clicks. Daily active usage went from 20% of the sales team to 85% within two weeks. The sales director called out in an all-hands that reps were now actually using their dashboards instead of logging in once each morning. One rep told me she appreciated that someone had finally built something that fit with how they worked, not how management thought they worked."

Landing: "That experience taught me to always check device analytics and observe a few real usage sessions. This approach has saved me from building several desktop-optimized features for teams that primarily work from their phones."

Mid-Level Example: E-commerce Checkout Redesign

Question: "Give me an example of when user needs conflicted with technical best practices."

Headline: "I fought to implement a 'messier' checkout process that actually helped customers to complete their purchases."

Key Point 1: "Our e-commerce site had a 35% cart abandonment rate that was hurting revenue. The existing checkout followed technical best practices with clean separation between shipping, billing, and payment steps. I convinced my manager to let me spend a week investigating why customers were dropping off. I analyzed session recordings and saw a pattern. International customers would get to the billing address step and abandon when they couldn't find their country's address format. Our 'clean' multi-step process meant they had already invested time before hitting this blocker."

Key Point 2: "I proposed consolidating everything into a single page with smart defaults. The engineering team pushed back hard. Single-page checkout meant complex state management, duplicate validation logic, and messier code. They were right from a technical perspective. But I'd learned that 50% of our abandonment happened between steps, not within them. I worked with our UX designer to create a version that showed all required information upfront while progressively revealing details. This was technically clunkier, but it matched how customers thought about checkout as one action, not four."

Key Point 3: "I A/B tested the new checkout with 10% of traffic. Cart abandonment dropped from 30% to 25% in the test group. That translated to roughly \$50,000 additional revenue per month. I also tracked engineering metrics. Page load time increased by 250ms. Admittedly, the code was more complex. I documented the trade-offs clearly for the team. When they saw the revenue impact, even the skeptics agreed that user needs should drive these types of decisions. We spent the next sprint cleaning up the implementation while keeping the improved checkout intact."

users."

Senior-Level Example: Healthcare Portal Simplification

Question: "Tell me about a time you advocated for users in a technical decision."

Headline: "I pushed to simplify our patient portal when the product team wanted to add advanced health tracking features."

Key Point 1: "I led the team modernizing our patient healthcare portal, which was used by 200K patients across several states. The product team kept pushing for advanced features such as symptom tracking, medication interaction warnings, and goal-setting. But our data showed that 65% of the patients couldn't even successfully book appointments through the portal. I spent a week analyzing user sessions and found that the patients were overwhelmed by the medical terminology and complex site navigation. Elderly patients, who made up 40% of our user base, were calling our support line instead of using the portal. I realized that the advanced features would help power users, but they would further alienate most of the other patients."

Key Point 2: "I partnered with our support team to listen to patient calls. Their frustration was heartbreaking. They just wanted to see their doctors, but they couldn't figure out our interface. I organized sessions so that engineers could watch elderly patients as they tried to book appointments. One 75-year-old patient spent 15 minutes looking for a 'schedule an appointment' button, but there was none, because we had labeled the button 'Appointment Management Portal.' Another wanted to book a video call with her doctor but didn't realize that was what we meant by 'Virtual Care Consultation.' I created a highlight reel of these sessions that showed how the terminology we had employed was impeding access to basic healthcare."

Key Point 3: "I convinced leadership to pause feature development and focus on simplification. We rewrote the entire interface using plain language. 'Appointment Management Portal' became 'Book a Visit.' We added a giant 'Need to See Your Doctor?' button on the homepage. We hid the advanced features behind a 'More Options' menu. The key was A/B testing every terminology change with real patients to ensure that our 'simpler' language was indeed clearer to them. Some surprises emerged. For example, 'Book Appointment' tested better than 'Book a Visit' with younger patients. Within six weeks, the percentage of patients who successfully booked appointments through the portal jumped from 35% to nearly 90%. Support call volume dropped by 40%."

Landing: "That portal simplification taught me that in healthcare technology, accessibility isn't optional. User advocacy sometimes means protecting users from feature creep. Now we always A/B test terminology changes with real users before rolling them out broadly."

Staff-Level Example: Cross-Platform Consistency

Headline: "Two years ago, our fitness app looked successful on each platform individually, but cross-platform inconsistencies were confusing users and killing retention."

Key Point 1: "Our fitness tracking application had grown to 75K active users across smartwatch, mobile, and web platforms. I noticed that the platform teams rarely talked to each other, and when I looked at the apps side by side, the user experience seemed to me to be fragmented. The watch app tracked 'Activities,' the mobile app called them 'Workouts,' and the web used 'Training Sessions.' The same fitness activity showed different calories burned on each platform due to different calculation methods. I suspected that this lack of consistency was confusing users, so I built analytics to track cross-platform user journeys. The data confirmed it: 6% of our total user base was churning because platforms didn't agree with each other, and support tickets and app reviews specifically cited inconsistent data. Our success on individual platforms had masked how badly we were failing users who expected a unified experience."

Key Point 2: "I organized research sessions where all platform teams watched the same users struggle with our inconsistencies. The siloed teams had never actually seen users try to use multiple platforms together. One marathon runner showed us three different weekly mileage totals across our platforms. She'd been maintaining her own spreadsheet because she couldn't trust our data. Another user had missed his workout streak achievement because the watch and phone apps calculated streaks differently. Engineers who'd been proud of their platform-specific features suddenly understood the trust we were breaking with our users. The confusion was so bad that users were warning each other in app store reviews to stick to just one platform."

Key Point 3: "Breaking down the silos was more than a technical challenge. I led the creation of unified data models and calculation methods, but also, with the support of the CEO, I established a new policy that measured success differently. Instead of platform-specific metrics, teams would be evaluated on cross-platform user retention. Any team that shipped a feature that broke cross-platform consistency had to make fixing it their top priority before moving to new work. No exceptions. The first time it happened, the mobile team had to delay their new social features by a week to align their calorie calculations with other platforms. Word spread quickly. Within a quarter, user retention had improved by 25%. Support tickets about data inconsistencies virtually disappeared. Reviews warning users to stick to one platform were replaced by praise for our consistent cross-device experience."

Landing: "Two years later, our consistent cross-platform experience has become our biggest competitive advantage against apps that stick to one platform. Users trust their data, whether they're checking their watch mid-run or analyzing trends on desktop. The change to our metrics of success transformed our siloed platform teams into a unified product organization."

If Examples Don't Come Easily

translate user needs into technical decisions. Have you ever discovered that users were struggling with something nobody had yet reported? That shows early awareness of the needs of users. Have you ever advocated for users even when it meant more work for your team? If you have made technical choices that have helped users succeed despite the inconvenience, you have shown strong customer focus skills.

Reflection Questions:

- When have you discovered user needs that have differed from stated requirements?
- What technical decisions have you made based on observing user behavior?
- Have you simplified complex features after understanding user workflows?
- When did you advocate for users even when it created more technical work?
- What user feedback have you gathered while managing regular development?
- Have you identified different user segments with conflicting needs?
- When have you measured whether your solution has actually helped users?
- What user problems have you solved that weren't stated in any requirements?
- Have you helped non-technical teams to better understand the needs of users?
- When has a deeper knowledge of users changed your entire approach?

Key Takeaways

 **Customer focus** means understanding real user needs beyond their stated requirements and translating those needs into effective technical solutions. It requires carefully balancing the conflicting needs of different user segments. And it means measuring success from the user's perspective, not just technical completion.

Strong Customer Focus Stories Include:

- Evidence of direct user research or observation.
- Clear connection between user needs and technical decisions.
- Examples of advocating for users when it required trade-offs.
- Measurement of actual user outcomes, not just feature delivery.
- Recognition of different user segments and their varied needs.

Avoid These Traps:

- Don't assume you know what users need without investigation.
- Don't describe building features without confirming they have helped users.
- Don't treat all users as having the same needs and abilities.
- Don't focus only on technical metrics without user outcomes.

☐ Mark as Complete

[Teach on ByteByteGo](#)

[Be an Affiliate](#)

[Become a Contributor](#)

hi@bytebytego.com

[Report a Bug](#)

[Our Team](#)

[Newsletter](#)

[Privacy Policy](#)

[Terms of Service](#)

[Do Not Sell or Share My Personal Information](#)

[YouTube](#)

[Visual Dev Guides](#)

[Find Jobs](#)

[Prepare for Coding Interviews](#)

Copyright ©2022-2026 ByteByteGo Inc.
All rights reserved.